Programming for Robotics: Project Report

Project Title: Forest Rover-OL8
GitHub link: https://github.com/CobberH/3003ICT_Group_OL8/
Group Members:

- Jacob Hennessy – s5342134
- Olivia Sealey – s5331855
- Jacob Stevens - s5409472

Specialisation Option Chosen: Autonomous Robot (Webots)

1. Project Overview

Forest Rover-OL8 is an autonomous search and rescue robot designed to locate missing persons within hazardous forest environments. The robot addresses the challenge of manually searching difficult terrain by autonomously navigating, detecting environmental hazards, mapping explored areas and identifying survivor markers using onboard sensors and intelligent decision making. The final controller integrates autonomous navigation, perception driven behaviour, obstacle avoidance and grid-based mapping using a Finite State Machine (FSM) architecture. The robot uses multiple onboard sensors to support autonomous operation:

- 8 proximity/LiDAR-style distance sensors for obstacle detection and collision avoidance
- An RGB camera for colour-based object detection and target identification
- Wheel encoders for odometry and position estimation during exploration
- GPS for precise grid-based mapping

The primary actuators used are the left and right wheel motors enabling differential drive steering, obstacle avoidance, hazard escape manoeuvres and target approach behaviour.

The system implements several intelligent behaviours including autonomous terrain exploration, obstacle and hazard avoidance, grid-based mapping, and survivor detection with coordinate logging. Additional behaviours include recovery when trapped, hazard classification using colour detection, and duplicate filtering to prevent repeated target detections.
The robot maintains internal memory structures for explored terrain, hazards, blocked regions and survivor locations to support autonomous navigation and decision making.

2. Technical Requirements Compliance

Inputs – The robot uses 8 proximity sensors for obstacle detection, an RGB camera for colour-based hazard and survivor detection, and a GPS module for position tracking and mapping.
Outputs – Left and right wheel motors provide movement and steering behaviours while console outputs display constant tracking data, detected hazards and survivor locations.
FSM – The robot is controlled using a Finite State Machine (FSM) with behaviour-based state transitions.
Behavioural states – Implemented states include SEARCH, SAFETY_NAV, RECOVERY, APPROACH_OBJECT and ALL_FOUND.
Multi-condition decision logic – State transitions depend on multiple sensor conditions, including obstacle proximity, colour detection, stuck detection and object confirmation thresholds.

Safety or fail-safe mechanism – Collision avoidance, hazard escape recovery, blocked terrain mapping and stuck recovery behaviours are implemented to improve safety.
Structured system architecture diagram – The system follows a Sense → Process → Decide → Act architecture model.
Navigation – The robot autonomously explores unknown terrain using GPS-assisted mapping and encoder-based odometry, grid-based mapping and exploration behaviours.
Obstacle avoidance – Proximity sensors and RGB camera detect nearby obstacles and hazards triggering SAFETY_NAV and RECOVERY behaviours.
Perception-driven decision making – The RGB camera classifies green survivor targets and red/blue hazards/obstacles assisting navigation choices and behavioural state transitions.

3. System Architecture

The system follows a Sense → Process → Decide → Act architecture model.
Sense: Proximity sensors detect obstacles while the RGB camera identifies coloured hazards and survivor targets. Wheel encoders and GPS provide odometry data for position estimation.
Process: Sensor data is analysed using colour detection, obstacle checking and grid-based mapping to track explored terrain, hazards and targets.
Decide: A Finite State Machine (FSM) selects behaviours such as SEARCH, SAFETY_NAV, RECOVERY and APPROACH_OBJECT based on sensor conditions.
Act: Differential wheel motors control robot movement for exploration, obstacle avoidance, recovery and target approach behaviours.

The robot continuously reads sensor data, processes environmental information, selects a behavioural state and applies motor outputs during each control cycle.

4. Behaviour Model

The robot uses a Finite State Machine (FSM) to control autonomous behaviour. The FSM was chosen because it provides clear state transitions, modular behaviour control and decision making for autonomous robotics tasks. The main behavioural states are:
SEARCH**:** Default exploration state where the robot navigates unknown terrain and updates the map.
SAFETY_NAV**:** Activated when proximity sensors detect nearby obstacles or hazards causing the robot to avoid collisions.
RECOVERY**:** Triggered when the robot becomes stuck or enters hazardous terrain and will perform reverse and turning functions to escape.
APPROACH_OBJECT**:** Activated when a potential survivor target is detected, allowing the robot to approach and confirm the object.
ALL_FOUND**:** Final state entered once all survivor targets have been located.

State transitions are triggered using multi-condition logic based on obstacle proximity sensor readings, colour detection from the RGB camera, target confirmation thresholds, stuck detection using odometry and number of targets successfully identified.

5. Perception / Sensor Processing

The robot uses multiples sensors at the same time to support autonomous navigation, mapping and decision making based on perception. Proximity sensors, wheel encoders, GPS and an RGB camera are continuously read during each loop.

The proximity sensors provide distance measurements used for obstacle detection, collision avoidance, hazard escape behaviour and stuck detection. Wheel encoders are used for odometry, and GPS data is used for grid-based mapping during exploration.

The RGB camera performs colour tracking and object classification using thresholds based on image processing with green objects representing survivor targets, red objects represent hazards and blue objects represent environmental hazards such as rivers or lakes.

Sensor fusion and multi-condition logic are used to improve decision making. This affects the confirmation of a survivor target by ensuring that the camera detects enough green colour coverage, the object is centred within the camera view, and the front proximity sensors confirm the robot is close enough to the object. Obstacle avoidance decisions combine proximity sensor readings with colour classification to determine whether the robot should avoid, recover or map a hazard. Processed sensor data directly affects behavioural state transitions with the FSM.

6. Safety and Robustness

The robot uses several safety and fail-safe mechanisms to improve autonomous operation in the forest environment. Collision avoidance is achieved using proximity sensors that continuously monitor nearby obstacles. When hazards or obstacles are detected, the robot enters the SAFETY_NAV state and performs avoidance manoeuvres using different wheel control.

Hazard protection is implemented using colour-based detection and blocked terrain mapping. Red and blue hazards are stored within the robot's internal mapping to help avoid dangerous areas during exploration. Fail-safe recovery behaviour is implemented through the RECOVERY state. If the robot becomes stuck of blocked while attempting movement, recovery logic triggers reverse and turning changes to escape.

Timeout protection is also used during target approach behaviour and general exploration. If the robot is in the SAFETY_NAV state for too long, it will move to the RECOVERY state and if the target is lost for too long, it will return to SEARCH mode.

7. Code Structure

The program is structured around a continuous robot control loop which repeatedly checks sensor data, processes environmental information, selects a behavioural state and applies motor outputs. The system has modular helper functions to improve behaviour management.

Key functions include sensor processing and colour detection functions for identifying hazards and survivor targets, odometry and position estimation functions using wheel encoder and GPS data, obstacle avoidance and recovery functions and grid mapping and duplicate filtering functions

The main control loop continuously does the following:

Reads sensor inputs, Updates robot position and mapping data, Processes hazards and target detections, Selects the appropriate FSM state, Applies wheel motor outputs

The program using Webots controller library for robot hardware access and the python math library for odometry, angle calculations and distance measurements.

8. Key Code Snippets

```python
# Decide state
if len(targets_found) >= TARGET_LIMIT:
    state = "ALL FOUND"

elif state == "RECOVERY" and recovery_mode is not None:
    state = "RECOVERY"

elif stuck_counter > STUCK_LIMIT:
    recovery_mode = "STUCK"
    recovery_step = 0
    state = "RECOVERY"

elif close_pending_object:
    state = "APPROACH_OBJECT"

elif left_obstacle or right_obstacle:
    state = "SAFETY_NAV"

elif pending_object is not None:
    state = "APPROACH_OBJECT"

else:
    state = "SEARCH"
```

```python
if brightness > 60 and g > r * 1.15 and g > b * 1.15:
    green_count += 1
    green_x_sum += x
```

```python
if green_center_x is not None:
    image_center = W / 2
    error = green_center_x - image_center

    CENTER_TOLERANCE = W * CENTER_TOLERANCE_RATIO

    # Check error and correct direction
    if abs(error) > CENTER_TOLERANCE:
        turn = 0.015 * error

        base = 0.20 * MAX_SPEED

        leftSpeed = base + turn
        rightSpeed = base - turn

        leftSpeed = max(-0.4 * MAX_SPEED, min(0.4 * MAX_SPEED, leftSpeed))
        rightSpeed = max(-0.4 * MAX_SPEED, min(0.4 * MAX_SPEED, rightSpeed))
    else:
        leftSpeed = APPROACH_SPEED
        rightSpeed = APPROACH_SPEED

else:
    leftSpeed = 0.25 * MAX_SPEED
    rightSpeed = 0.25 * MAX_SPEED
```

The robot's camera and proximity sensors are used to determine the robot's current FSM state. During normal operation the robot remains in the SEARCH state until either an obstacle or survivor target is detected, causing transitions into SAFETY_NAV or APPROACH_OBJECT. When approaching a green target, image processing is used to determine the target's position within the camera frame, allowing wheel speeds to be continuously adjusted to keep the target centred while approaching it.

9. Results / Demonstration Summary

The robot locates a survivor using RGB and approaches slowly toward them, and once close enough it marks them with GPS coordinates to avoid doubling up on the same survivor. The mapping function provides an estimated map of the explored area to assist in rescue. When dealing with obstacles the robot would avoid them effectively.
Challenges and limitations: The robot would have its search speed hindered in areas dense with hazards, obstacles and walls e.g. following along a wall for a long period of time. Which caused the robot to spend a long time in the "SAFETY_NAV" state, until it escapes the area. The robot would also revisit locations if the area wasn't fully scanned.

10. Individual Contributions

Member	Contribution
Jacob Hennessy	FSM, Survivor, and hazard logic, mapping, Video demonstration and testing
Olivia Sealey	FSM, obstacle and camera logic, FSM diagram, video demonstration and testing
Jacob Stevens	FSM, GPS tracking and mapping logic, Video demonstration and testing.